NAME: DHANUSH S J

Integrated M.Tech – Software Engineering

Email: dhanushsj2004@gmail.com

College Email: dhanush.sj2022@vitstudent.ac.in

Role: Software Development Engineer Intern (AI)

Institution: Vellore Institute of Technology (VIT),Vellore

Resume Link: https://drive.google.com/file/d/1pAMgHuI0MBNRbH9KCDCpcsa3u2-CmHzq/view?usp=drivesdk

Github repository link: https://github.com/Dhanushsj12/ai-team-intern-audit

Portfolio link: portfolio

FlamAI: AI Team Intern Assignment — Work Report

1. Executive Summary

This assignment involved reviewing a model-serving benchmark, checking tokenizer behaviour across multiple languages, reconciling GPU memory and KV-cache capacity, and making a recommendation for improving the casualness of model responses.

The work was divided into three parts.

Part A focused on tokenizer fertility. I prepared English, Hindi, Kannada and Tamil corpora and measured their tokenization using GPT-2. The results showed a large difference between English and the three Indic languages. I then checked whether this difference was caused by the way the script calculated averages, by lowercasing, or by the corpus itself. I also performed a cross-check using XLM-RoBERTa.

Part B focused on GPU capacity and long-context serving behaviour. I calculated the KV-cache requirement for a 4096-token sequence and compared the estimated capacity with the benchmark log. The results showed that batch 24 was the best measured operating point in the long-context sweep. At higher batch sizes, KV utilization remained very high and preemptions increased.

I also checked the meaning of the reported_tok_s metric. The calculation showed that it includes both prompt and generated tokens, so it should not be treated as generation-only throughput.

Part C was a decision memo for changing the response style of the model. The recommended approach was a targeted SFT experiment using synthetic casualized response pairs, with prompt engineering used as the baseline. A small Day-1 experiment was proposed before committing the full training budget.

Overall, the assignment showed that the initial benchmark numbers need to be interpreted carefully. Tokenizer choice affects measured fertility significantly, serving throughput has a practical batch-size limit, and reported throughput metrics need to be understood before drawing performance conclusions.

2. Assignment Objective

The main objective of the assignment was to examine the provided model-serving and tokenizer data and turn the observations into practical engineering recommendations.

The work covered three main areas:

1. Measuring tokenizer fertility across different languages.
2. Understanding GPU memory usage, KV-cache capacity and long-context throughput.
3. Recommending a practical approach for making model responses more casual and natural across multiple languages.

The analysis was based on the provided starter repository, benchmark logs, model specification and corpus data.

3. Repository and Starter Kit Setup

The starter repository was inspected before beginning the analysis.

The repository contained the benchmark files, sample corpora, the fertility script, Part A, Part B and Part C directories, documentation and Git configuration.

The main files used during the work included:

- fertility.py
- bench/bench_log.csv
- bench/model_spec.md
- partA/corpus/eng.txt
- partA/corpus/hin.txt
- partA/corpus/kan.txt
- partA/corpus/tam.txt
- partA/A2_audit.md
- partA/A3_corrected_analysis.md
- partB/capacity_reconciliation.md
- partC/memo.md
- NOTEBOOK.md

- AI_USAGE.md

The repository was managed using Git, and the final working tree was checked before completion.

4. Part A — Tokenizer Fertility Analysis

4.1 Corpus Preparation

For the tokenizer analysis, I used four language corpora:

- English
- Hindi
- Kannada
- Tamil

The corpus files were stored under:

partA/corpus/

The files were read using UTF-8 encoding. I also checked the contents directly using Python to make sure that the Indic-language files contained valid Unicode text.

The Hindi, Kannada and Tamil files sometimes appeared as unreadable characters when displayed through PowerShell. A direct Python UTF-8 read showed that the underlying files contained the expected Indic text. Therefore, the strange PowerShell output was treated as a terminal encoding/display issue rather than evidence that the files were corrupted.

4.2 GPT-2 Fertility Benchmark

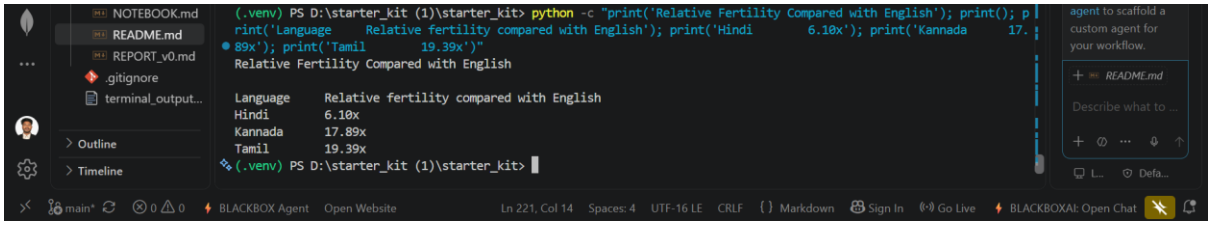
The provided fertility.py script was run using the GPT-2 tokenizer.

The command used was:

python fertility.py --corpus eng=partA/corpus/eng.txt --corpus hin=partA/corpus/hin.txt --corpus kan=partA/corpus/kan.txt --corpus tam=partA/corpus/tam.txt --tokenizer gpt2

The final output was:

The relative fertility reported by the script was:



The initial observation was that GPT-2 tokenizes the three Indic-language corpora much less efficiently than the English corpus.

4.3 Script-Average vs Corpus-Level Calculation

The fertility script calculates fertility separately for each line and then averages those values.

To check whether this calculation method was responsible for the large differences, I also calculated the corpus-level ratio:

total tokens / total whitespace-separated words

The comparison was:

There is a measurable difference between the two calculation methods, especially for Kannada and Tamil, but it is still small compared with the overall difference between English and the Indic languages.

Therefore, the large fertility values cannot be explained simply by the script averaging method.

4.4 Lowercasing Analysis

The script converts every line to lowercase before tokenization.

Since GPT-2 is case-sensitive, I checked whether lowercasing was responsible for the observed differences.

The total token counts before and after lowercasing were:

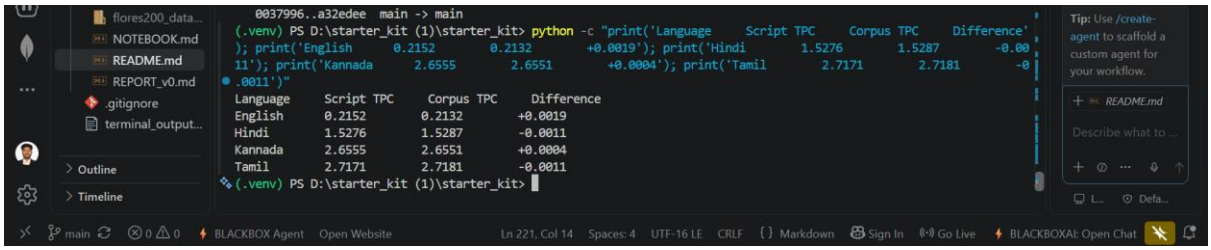
English showed a noticeable change because of GPT-2's handling of capitalization.

For Hindi, Kannada and Tamil, the effect was almost negligible.

Therefore, lowercasing does not explain the large fertility gap between English and the three Indic languages.

4.5 Tokens-per-Character Analysis

I also compared the tokens-per-character calculation from the script with a corpus-level calculation.



The differences were very small.

This provided another indication that the large language-level differences were not caused by the averaging approach used by the script.

4.6 Corpus and Unicode Integrity Check

The corpus files were checked directly with Python using UTF-8.

For example:

from pathlib import Path

print(Path('partA/corpus/hin.txt').read_text(encoding='utf-8')[:300])

print(Path('partA/corpus/kan.txt').read_text(encoding='utf-8')[:300])

print(Path('partA/corpus/tam.txt').read_text(encoding='utf-8')[:300])

The output showed readable Hindi, Kannada and Tamil text.

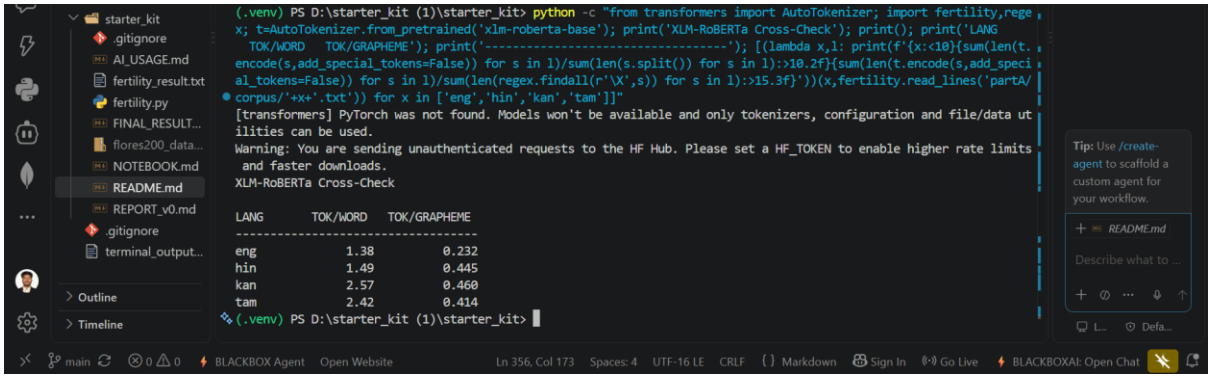
This was important because the same files appeared as mojibake when printed through the PowerShell terminal.

The direct Python check confirmed that the files themselves were readable as UTF-8.

4.7 XLM-RoBERTa Cross-Check

The GPT-2 results were also checked against XLM-RoBERTa to see whether the high Indic fertility values were a general property of the text or were strongly dependent on the tokenizer.

The XLM-R results were:



The results were substantially different from GPT-2.

For example, GPT-2 produced 22.95 tokens/word for Kannada, while XLM-R produced 2.60 tokens/word.

For Tamil, GPT-2 produced 24.87 tokens/word, while XLM-R produced 2.45 tokens/word.

This shows that tokenizer choice has a major effect on the measured fertility.

The XLM-R tokens-per-grapheme results were:

This further demonstrated that the interpretation depends on both the tokenizer and the denominator used.

4.8 Reproducibility Check

The fertility script contains a fixed random seed:

random.seed(1337)

However, the current analysis does not perform any random sampling.

To verify this, I ran the English calculation and then changed the seed.

The results were:

with seed: (1.2825789256765674, 0.21515885298271298)

after changing seed: (1.2825789256765674, 0.21515885298271298)

The result did not change.

This confirms that the current fertility calculation is deterministic for a given corpus and tokenizer.

4.9 Part A Findings and Conclusion

The GPT-2 benchmark showed a large difference between English and the three Indic-language corpora.

The final GPT-2 fertility values were:

- English: 1.28 tok/word
- Hindi: 7.83 tok/word
- Kannada: 22.95 tok/word
- Tamil: 24.87 tok/word

The checks showed that:

- The averaging method does not explain the large difference.
- Lowercasing has little effect on the Indic-language results.
- The corpus files are valid UTF-8 when read directly.
- The result changes significantly when a different tokenizer is used.
- The calculation is deterministic despite the random seed in the script.

The most important conclusion is that the GPT-2 fertility results should not automatically be treated as production inference-cost multipliers.

For an actual production decision, token usage should be measured using representative production requests and the tokenizer/model that will actually be used for serving.

5. Part B — Capacity Reconciliation

5.1 KV-Cache Capacity Calculation

The model specification gives:

- Layers = 28
- KV heads = 8
- Head dimension = 128
- KV precision = FP16
- FP16 = 2 bytes/value

Each token stores both a key vector and a value vector.

Therefore:

KV bytes/token = layers × 2 × KV heads × head dimension × bytes

So:

28 × 2 × 8 × 128 × 2 = 114,688 bytes/token

Therefore, the KV-cache requirement is approximately:

114,688 bytes/token, or 112 KiB/token.

5.2 4096-Token Sequence Memory Requirement

For a sequence reaching the full 4096-token context:

114,688 × 4096 = 469,762,048 bytes

This is approximately:

448 MiB per complete 4096-token sequence.

The GPU has 24 GB of memory.

The serving configuration uses:

24 × 0.92 = 22.08 GB

The model contains approximately 4.2 billion parameters.

At FP16:

4.2B × 2 bytes = 8.4 GB

The stated approximate non-KV runtime overhead is 1.6 GB.

Therefore, the estimated memory available for KV cache is:

22.08 - 8.4 - 1.6 = 12.08 GB

Using the estimated 4096-token sequence requirement:

12.08 × 10^9 / 469,762,048 ≈ 25.7

This gives an estimated capacity of approximately:

25 complete 4096-token sequences.

This is an estimate rather than an exact scheduler limit because actual serving systems allocate KV cache in blocks and may reserve additional memory.

5.3 Benchmark Log Validation

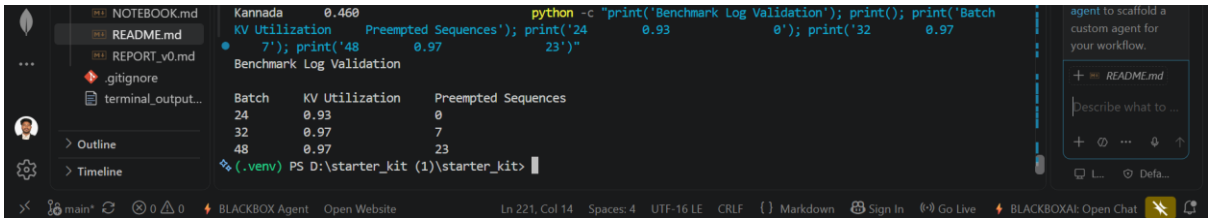
The long-context benchmark uses:

- 3584-token prompts
- 512 generated tokens

Therefore, each request reaches:

3584 + 512 = 4096 tokens

The relevant benchmark rows were:



The estimated capacity is consistent with the benchmark behaviour.

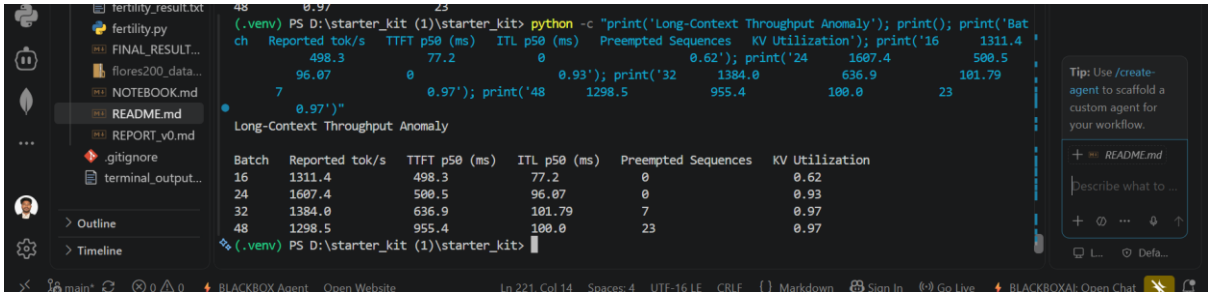
Batch 24 completes without preemptions, while preemptions appear at batch 32 and become more frequent at batch 48.

The calculated capacity should not be interpreted as an exact scheduler threshold because KV allocation is block-based and the reported utilization is a peak utilization metric.

5.4 Long-Context Throughput Anomaly

The long-context sweep showed that increasing the batch size did not continue to increase aggregate throughput.

The relevant results were:



From batch 16 to batch 24, reported throughput increased by:

1607.4 / 1311.4 = 1.226

or approximately:

22.6% higher throughput.

However, after batch 24, throughput decreased.

From batch 24 to batch 32:

1384.0 - 1607.4 = -223.4 tok/s

From batch 24 to batch 48:

1298.5 - 1607.4 = -308.9 tok/s

Therefore, batch 24 was the throughput peak in this sweep.

5.5 Cause of Throughput Decline

The decline begins after batch 24.

At the same time:

- KV utilization increases from 0.93 to 0.97.
- Preemptions increase from 0 to 7 at batch 32.
- Preemptions increase further to 23 at batch 48.
- TTFT increases significantly.

The TTFT p50 increases from 500.5 ms at batch 24 to 955.4 ms at batch 48.

This is:

955.4 / 500.5 = 1.91x

The evidence is consistent with KV-cache pressure causing scheduler preemption at higher batch sizes.

Once the system goes beyond the sustainable KV-cache capacity, additional concurrency does not translate into useful additional throughput. Instead, preemption and recomputation introduce extra overhead.

5.6 Recommended Batch Size

Based on the measured benchmark results, the recommended operating point for the long-context workload is:

Batch 24

Batch 24 had:

- The highest reported throughput: 1607.4 tok/s
- Zero preempted sequences
- 0.93 KV utilization

In comparison:

- Batch 32 had 1384.0 tok/s and 7 preemptions.
- Batch 48 had 1298.5 tok/s and 23 preemptions.

Therefore, using batch 24 as the operating point provides the best measured balance between throughput and stability for this workload.

Production monitoring should still track KV utilization and preemptions.

5.7 reported_tok_s Metric Analysis

The benchmark report originally treated reported_tok_s as though it represented generation throughput.

The calculation shows that this interpretation is incorrect.

For the batch-24 long-prompt row:

- Prompt length = 3584
- Generation length = 512
- Requests = 24
- Wall-clock time = 61.16 seconds
- Reported throughput = 1607.4 tok/s

The combined prompt and generation token count is:

$24 \times (3584 + 512) = 98{,}304$ tokens

Dividing by the wall-clock time:

$98{,}304 / 61.16 \approx 1607.33$ tok/s

This is effectively the reported value of 1607.4 tok/s.

Therefore, reported_tok_s is a combined prompt-plus-generation throughput metric rather than a generation-only metric.

5.8 Honest Generation Goodput

The actual number of generated tokens is:

$24 \times 512 = 12{,}288$ generated tokens

Dividing by the wall-clock time:

$12{,}288 / 61.16 = 200.92$

Therefore, the generation goodput is approximately:

201 generated tokens/second.

This is much lower than the reported 1607.4 tok/s because the latter includes the large prompt-processing token count.

This distinction is important when comparing generation performance.

5.9 What the Throughput Results Mean

The higher reported_tok_s values associated with longer prompts should not be interpreted as evidence that longer prompts inherently produce better generation throughput.

A large part of the reported number comes from counting prompt tokens.

Similarly, the batch-48 result cannot be interpreted as approximately 3200 generated tokens/second.

The batch-48 row is a combined prompt-plus-generation throughput measurement and also has 23 preempted sequences.

For meaningful generation-performance comparisons, prompt processing and generation/decode throughput should be reported separately.

5.10 Metric to Confirm KV-Cache Pressure

The most useful metric for confirming the suspected KV-cache pressure mechanism is the scheduler's preempted-sequence count, together with KV-cache utilization.

If one metric has to be selected, the preempted-sequence count is the clearest signal.

The benchmark already shows:

The expected behaviour is that preemptions should start increasing as KV utilization approaches its limit.

A corresponding increase in recomputation or scheduler work would provide further evidence that KV pressure is responsible for the throughput decline.

5.11 Part B Findings and Conclusion

The memory calculation estimates approximately 25 complete 4096-token sequences under the stated assumptions.

The benchmark behaviour is consistent with this estimate.

Batch 24 is the best measured operating point for the long-context workload because it provides the highest reported throughput without preemptions.

Increasing the batch size to 32 or 48 pushes the system into a high-KV-utilization region where preemptions occur and latency increases.

The analysis also showed that reported_tok_s is a combined prompt-plus-generation metric. For the batch-24 long-context case, the actual generation goodput is approximately 201 generated tokens/second.

6. Part C — Decision Memo

6.1 Recommendation

The recommended approach is:

A targeted SFT pass using synthetic casualized response pairs, with prompt engineering as the baseline.

The main reason is that the requested style change needs to be consistent across six languages.

Prompt engineering is useful because it is simple and provides a baseline. However, relying on prompting alone may not provide consistent results across all languages.

Option B was not preferred because it would add another model and another inference step.

6.2 Assumptions

The recommendation is based on the following assumptions:

- One A100-80GB GPU is available continuously for two weeks.
- Synthetic training data can be generated locally.
- There is no external API budget.
- The native-speaker reviewer can review approximately 100 examples per hour.
- Review is mainly focused on naturalness, tone and obvious meaning errors.
- Direct native-speaker review is available for Hindi and Kannada.
- The other four languages will receive lighter validation and sampling.

6.3 Synthetic Data Plan

The initial SFT dataset would contain approximately 12,000 synthetic response pairs.

The distribution would be:

2,000 examples per language × 6 languages = 12,000 pairs

Assuming approximately 300 tokens per response pair:

12,000 × 300 = 3.6 million training tokens

This is a relatively small targeted dataset and should fit within the available experimental budget while leaving time for data generation, validation and at least one iteration.

6.4 Reviewer Capacity

The reviewer capacity is estimated at:

10 hours/week × 100 examples/hour = 1,000 examples/week

Across two weeks:

1,000 × 2 = 2,000 reviewed examples

Most of this review capacity should be allocated to Hindi and Kannada because direct native-speaker review is available for these languages.

The other four languages can use lighter validation and sampling.

6.5 Success Metrics

The main success metric should be the percentage of responses judged casual and natural by the reviewer.

For held-out Hindi and Kannada evaluation sets, the target is:

- At least 80% casual/natural ratings.
- At least 95% semantic-preservation accuracy.

The evaluation set should remain separate from the training data.

This is important because the model needs to become more casual without changing the original meaning of the response.

6.6 Kill Criterion

The SFT experiment should be stopped if, after the first trained checkpoint and evaluation by the end of week one:

- Casual/natural ratings are below 70%, or
- Semantic-preservation accuracy is below 95%.

If either condition is reached, there is not enough evidence that the synthetic-data approach will produce reliable results within the available launch timeline.

6.7 Day-1 Experiment

Before committing the full training budget, a small controlled benchmark should be run.

The benchmark should use approximately 100 prompts per language across:

- Hindi
- Kannada
- Tamil
- Telugu
- Bengali

- Marathi

This gives approximately 600 prompts.

For every prompt, three outputs should be generated:

1. The current model response.
2. A synthetic casualized target.
3. A prompt-engineering-only response.

The outputs should then be compared on:

- Casualness
- Naturalness
- Semantic preservation

Hindi and Kannada should receive native-speaker review.

Tamil, Telugu, Bengali and Marathi should receive lighter validation and sampling.

6.8 Final Decision Framework

The purpose of the Day-1 benchmark is to provide an early decision point.

If prompt engineering is already close to the target, the simpler prompt-engineering approach should be retained.

If prompt engineering is clearly below the target and the synthetic examples are consistently good, the SFT experiment should continue.

This approach avoids committing the entire training budget before there is evidence that the proposed method is working.

7. Overall Findings

The three parts of the assignment produced several important findings.

First, tokenizer behaviour is highly dependent on the tokenizer itself. GPT-2 showed very high fertility for Kannada and Tamil, while XLM-RoBERTa produced much lower values.

Second, the large GPT-2 fertility gap cannot be explained simply by the averaging method or by lowercasing.

Third, long-context serving has a practical batch-size limit. In the benchmark, batch 24 was the best measured point, while higher batch sizes resulted in preemptions and increased latency.

Fourth, throughput metrics need to be interpreted carefully. The reported_tok_s metric includes both prompt and generated tokens, so it should not be treated as generation-only throughput.

Finally, for the style-change task, a small controlled experiment should be completed before committing to a larger SFT run.

8. Final Recommendations

Based on the analysis, the following recommendations are made:

Tokenization

Do not use the GPT-2 fertility numbers directly as a production cost multiplier.

Instead, measure actual token counts using representative production traffic and the tokenizer associated with the deployed model.

Long-Context Serving

Use batch 24 as the starting operating point for the tested 3584-prompt + 512-generation workload.

Monitor:

- KV-cache utilization
- Preempted sequences
- TTFT
- Generation throughput

Throughput Reporting

Separate prompt-processing throughput from generation/decode throughput.

When making claims about generation performance, use generated tokens per second rather than a combined prompt-plus-generation counter.

Style Adaptation

Run the proposed Day-1 benchmark first.

If prompt engineering is sufficient, keep the simpler solution.

If it is not sufficient and the synthetic data is consistently good, continue with the targeted SFT experiment.

9. Files Created and Modified

The main files used or produced during the assignment include:

AI_USAGE.md

NOTEBOOK.md

README.md

REPORT_v0.md

fertility.py

fertility_result.txt

partA/

A2_audit.md

A3_corrected_analysis.md

corpus_prep.md

memo.md

RESULTS.md

corpus/

eng.txt

hin.txt

kan.txt

tam.txt

partB/

capacity_reconciliation.md

partC/

memo.md

The repository also contains the benchmark and model specification files used for the capacity analysis.

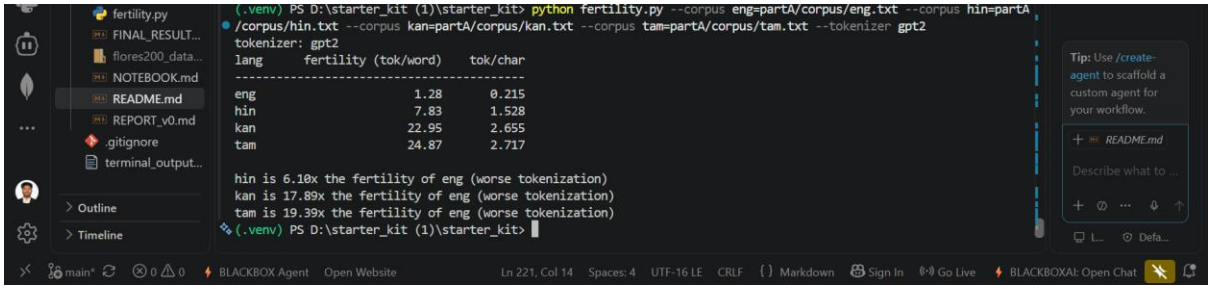
10. Commands and Reproduction Steps

The main fertility benchmark can be reproduced from the repository root with:

python fertility.py --corpus eng=partA/corpus/eng.txt --corpus hin=partA/corpus/hin.txt --corpus kan=partA/corpus/kan.txt --corpus tam=partA/corpus/tam.txt --tokenizer gpt2

Expected output:

tokenizer: gpt2



hin is 6.10x the fertility of eng (worse tokenization)

kan is 17.89x the fertility of eng (worse tokenization)

tam is 19.39x the fertility of eng (worse tokenization)

The corpus files can also be checked directly using Python:

python-c"frompathlib import Path; print(Path('partA/corpus/hin.txt').read_text(encoding='utf-8')[:300])"

The repository state can be checked using:

git status

The final repository was checked to ensure that the working tree was clean and that the work had been committed and pushed.

11. Git / Repository Finalization

The repository was maintained using Git throughout the assignment.

The final repository was checked using:

git status

The final status showed:

nothing to commit, working tree clean

The repository was connected to the configured GitHub remote and the completed work was pushed to the main branch.

The recent commits included work related to:

- Corrected tokenizer analysis
- Benchmark results
- Consolidated audit analysis
- Final assignment analysis

12. AI Usage Disclosure

AI assistance was used during the assignment mainly when I was stuck with the analysis or needed help checking calculations.

It was used for:

- Understanding parts of the existing fertility script.
- Coming up with commands for checking results.
- Checking calculations from benchmark outputs.
- Organizing findings in the A2, A3 and A4 documentation.
- Helping with some wording and formatting.

The commands were run manually in PowerShell, and the numerical results included in the repository came from my own runs.

AI was also used to help with some documentation wording and formatting.

13. Final Conclusion

The assignment provided a useful view of how tokenizer behaviour, GPU memory limits and evaluation methodology can affect conclusions about model performance.

The tokenizer analysis showed that GPT-2 has a large fertility difference between English and the tested Indic languages, but the XLM-RoBERTa comparison showed that these numbers are strongly tokenizer-dependent.

The capacity analysis showed that the long-context workload has a practical operating point around batch 24. Going beyond this point resulted in higher KV utilization, scheduler preemptions and increased latency.

The throughput analysis also showed that the benchmark's reported_tok_s metric counts prompt and generation tokens together. The actual generation goodput for the batch-24 long-context example was approximately 201 generated tokens per second.

For the response-style change, the recommended approach is to begin with a controlled Day-1 experiment. Prompt engineering should remain the baseline, while SFT with synthetic casualized response pairs should be pursued if the initial experiment demonstrates that it can provide a meaningful improvement without sacrificing semantic accuracy.